

Experiment 2: Exploratory Data Analysis (EDA), Descriptive Statistics, and Data Visualization Using Python

Mapped CO(s): CO1, CO2

Aim

To perform Exploratory Data Analysis (EDA), descriptive statistical analysis, and data visualization on the **Titanic dataset** using Python.

Objectives

- To explore the structure of the dataset.
- To calculate descriptive statistics for numerical variables.
- To identify missing values and understand data distribution.
- To visualize the data using different graphs.
- To interpret patterns and trends from the dataset.

Prerequisites

- Basic knowledge of Python programming.
- Basic understanding of statistics.
- Python 3.x installed.
- Jupyter Notebook / Google Colab.
- Required libraries: Pandas, NumPy, Matplotlib, and Seaborn.

Theory / Background

Exploratory Data Analysis (EDA) is the process of examining a dataset to understand its structure, detect patterns, identify missing values, and discover relationships between variables before applying machine learning or statistical models.

Descriptive statistics summarize the main characteristics of data using numerical measures such as mean, median, mode, standard deviation, minimum, maximum, and quartiles. These measures help understand the distribution and variability of the dataset.

Data visualization represents data graphically, making it easier to identify trends, patterns, and outliers. Python libraries such as Matplotlib and Seaborn provide various visualization techniques including histograms, bar charts, box plots, scatter plots, and heatmaps.

EDA helps improve data quality and supports better decision-making by providing meaningful insights into the dataset.

Dataset Description

Dataset Name: Titanic Dataset

Source: Seaborn / Kaggle

The Titanic dataset contains demographic and travel information of passengers aboard the RMS Titanic. It includes passenger details such as age, gender, passenger class, fare, and survival status, making it suitable for practicing exploratory data analysis and visualization techniques.

Important Attributes

Attribute	Description
survived	Survival status (0 = No, 1 = Yes)
pclass	Passenger class
sex	Gender of passenger
age	Age of passenger
fare	Ticket fare
class	Travel class
embarked	Port of embarkation
sibsp	Number of siblings/spouses aboard
parch	Number of parents/children aboard

Step 1: Import Required Libraries

We import Pandas for data handling, NumPy for numerical operations, and Matplotlib/Seaborn for visualization.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Display settings
pd.set_option('display.max_columns', None)
sns.set(style="whitegrid")
%matplotlib inline

print("Libraries imported successfully.")

Libraries imported successfully.
```

Step 2: Load the Titanic Dataset

The Titanic dataset is directly available through Seaborn's built-in dataset repository, so we load it using `sns.load_dataset('titanic')`.

```
# Load the Titanic dataset from Seaborn's built-in datasets
titanic = sns.load_dataset('titanic')
```

```
# Display the first 5 records
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

Step 3: Display Dataset Information

We check the shape, column names, data types, and general structure of the dataset.

```
# Shape of the dataset (rows, columns)
print("Shape of dataset:", titanic.shape)
```

```
# Column names
print("\nColumn names:\n", titanic.columns.tolist())
```

Shape of dataset: (891, 15)

Column names:

```
['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',  
'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',  
'alive', 'alone']
```

```
# Detailed information about the dataset (data types, non-null counts)
titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   survived        891 non-null   int64
1   pclass          891 non-null   int64
2   sex             891 non-null   object
3   age            714 non-null   float64
4   sibsp          891 non-null   int64
5   parch          891 non-null   int64
6   fare           891 non-null   float64
7   embarked        889 non-null   object
8   class          891 non-null   category
9   who            891 non-null   object
10  adult_male      891 non-null   bool
11  deck           203 non-null   category
12  embark_town     889 non-null   object
13  alive          891 non-null   object
14  alone          891 non-null   bool
```

```
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

Step 4: Check for Missing Values

Identifying missing values is a crucial step in EDA since missing data can affect statistical analysis and model performance.

```
# Count of missing values in each column
missing_values = titanic.isnull().sum()
print("Missing values per column:\n")
print(missing_values)
```

Missing values per column:

survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0

```
deck          688
embark_town    2
alive         0
alone         0
dtype: int64
```

```
# Percentage of missing values per column
```

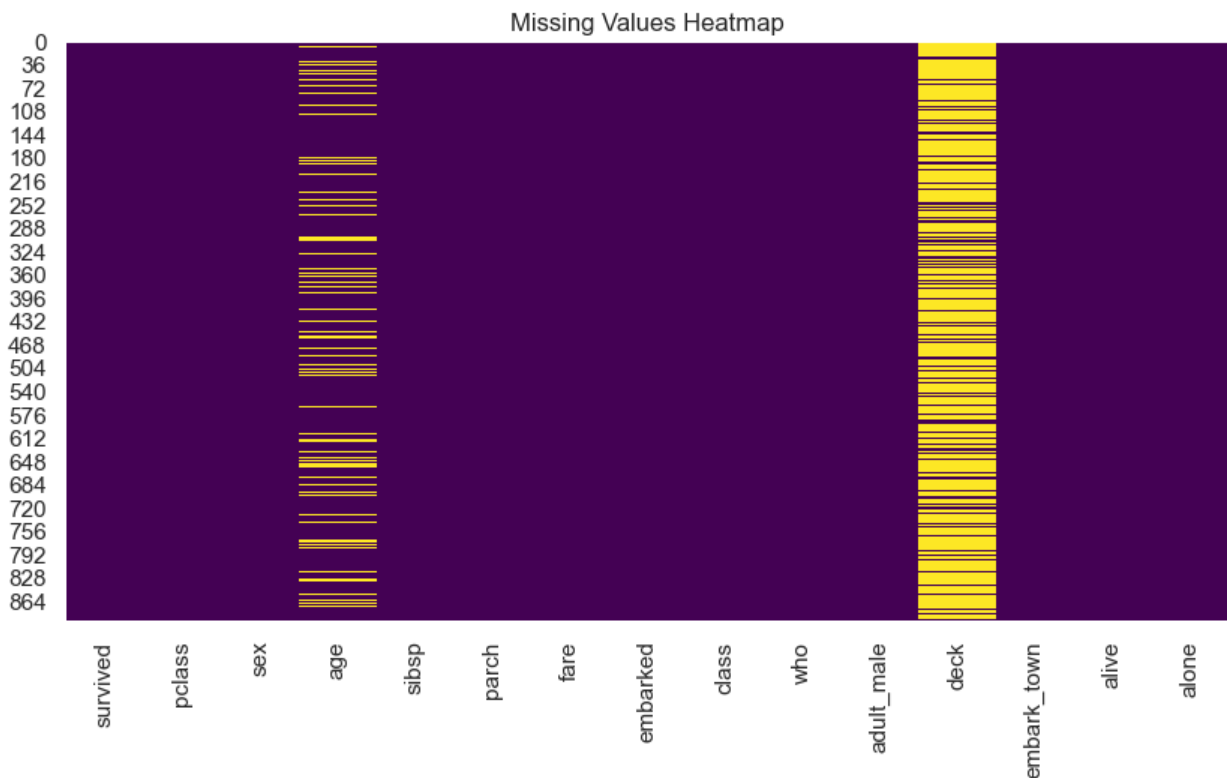
```
missing_percent = (titanic.isnull().sum() / len(titanic)) * 100
missing_percent = missing_percent[missing_percent >
0].sort_values(ascending=False)
print("Percentage of missing values:\n")
print(missing_percent)
```

Percentage of missing values:

```
deck          77.216611
age           19.865320
embarked       0.224467
embark_town    0.224467
dtype: float64
```

```
# Visualizing missing values using a heatmap
```

```
plt.figure(figsize=(10, 5))
sns.heatmap(titanic.isnull(), cbar=False, cmap='viridis')
plt.title("Missing Values Heatmap")
plt.show()
```



Step 5: Descriptive Statistics

We use the `describe()` function to calculate summary statistics (mean, median, standard deviation, min, max, quartiles) for numerical variables.

```
# Descriptive statistics for numerical columns
titanic.describe()
```

	survived	pclass	age	sibsp	parch
fare					
count	891.000000	891.000000	714.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594
std	0.486592	0.836071	14.526497	1.102743	0.806057
min	0.000000	1.000000	0.420000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000
50%	0.000000	3.000000	28.000000	0.000000	0.000000
75%	1.000000	3.000000	38.000000	1.000000	0.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000

```
# Descriptive statistics including categorical columns
titanic.describe(include='all')
```

	survived	pclass	sex	age	sibsp
parch \					
count	891.000000	891.000000	891	714.000000	891.000000
unique	NaN	NaN	2	NaN	NaN
top	NaN	NaN	male	NaN	NaN
freq	NaN	NaN	577	NaN	NaN
mean	0.383838	2.308642	NaN	29.699118	0.523008
std	0.486592	0.836071	NaN	14.526497	1.102743
min	0.000000	1.000000	NaN	0.420000	0.000000
25%	0.000000	2.000000	NaN	20.125000	0.000000
50%	0.000000	3.000000	NaN	28.000000	0.000000

```

75%      1.000000      3.000000      NaN      38.000000      1.000000
0.000000
max       1.000000      3.000000      NaN      80.000000      8.000000
6.000000

```

```

      fare embarked  class  who adult_male deck  embark_town
alive \
count      891.000000      889      891      891      891      203      889
891
unique      NaN      3      3      3      2      7      3
2
top      NaN      S  Third  man      True      C  Southampton
no
freq      NaN      644      491      537      537      59      644
549
mean      32.204208      NaN      NaN      NaN      NaN      NaN      NaN
NaN
std      49.693429      NaN      NaN      NaN      NaN      NaN      NaN
NaN
min      0.000000      NaN      NaN      NaN      NaN      NaN      NaN
NaN
25%      7.910400      NaN      NaN      NaN      NaN      NaN      NaN
NaN
50%      14.454200      NaN      NaN      NaN      NaN      NaN      NaN
NaN
75%      31.000000      NaN      NaN      NaN      NaN      NaN      NaN
NaN
max      512.329200      NaN      NaN      NaN      NaN      NaN      NaN
NaN

```

```

      alone
count      891
unique      2
top      True
freq      537
mean      NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN

```

```

# Individual statistical measures for the 'age' column
print("Mean Age:", titanic['age'].mean())
print("Median Age:", titanic['age'].median())
print("Mode Age:", titanic['age'].mode()[0])
print("Standard Deviation of Age:", titanic['age'].std())
print("Minimum Age:", titanic['age'].min())
print("Maximum Age:", titanic['age'].max())

```

```
Mean Age: 29.69911764705882
Median Age: 28.0
Mode Age: 24.0
Standard Deviation of Age: 14.526497332334042
Minimum Age: 0.42
Maximum Age: 80.0
```

Step 6: Visualizing Categorical Variables

Bar charts are used to visualize the frequency distribution of categorical variables such as `sex`, `pclass`, `embarked`, and `survived`.

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

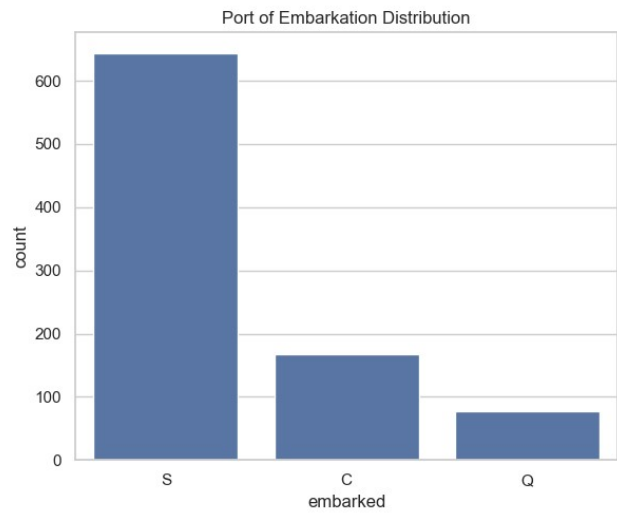
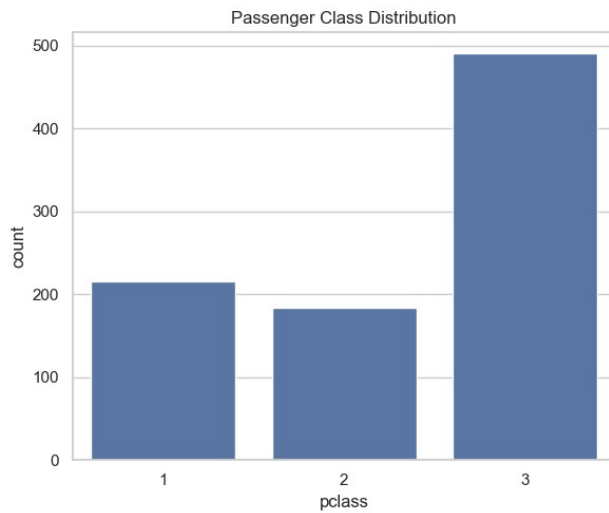
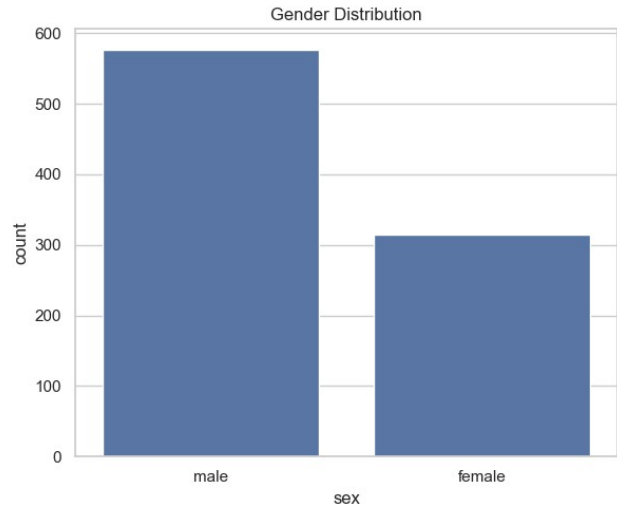
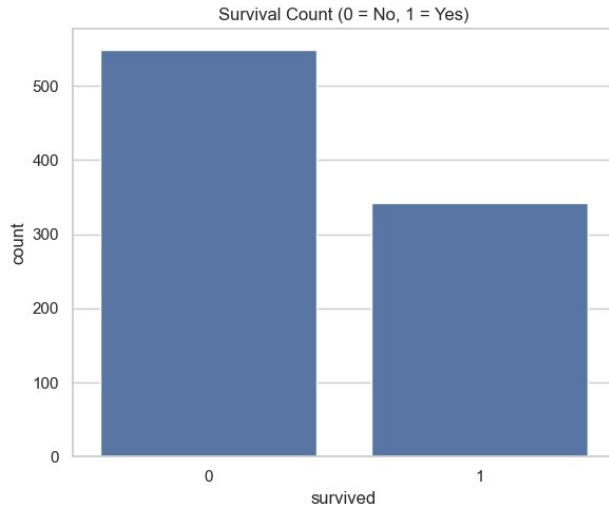
sns.countplot(x='survived', data=titanic, ax=axes[0, 0])
axes[0, 0].set_title('Survival Count (0 = No, 1 = Yes)')

sns.countplot(x='sex', data=titanic, ax=axes[0, 1])
axes[0, 1].set_title('Gender Distribution')

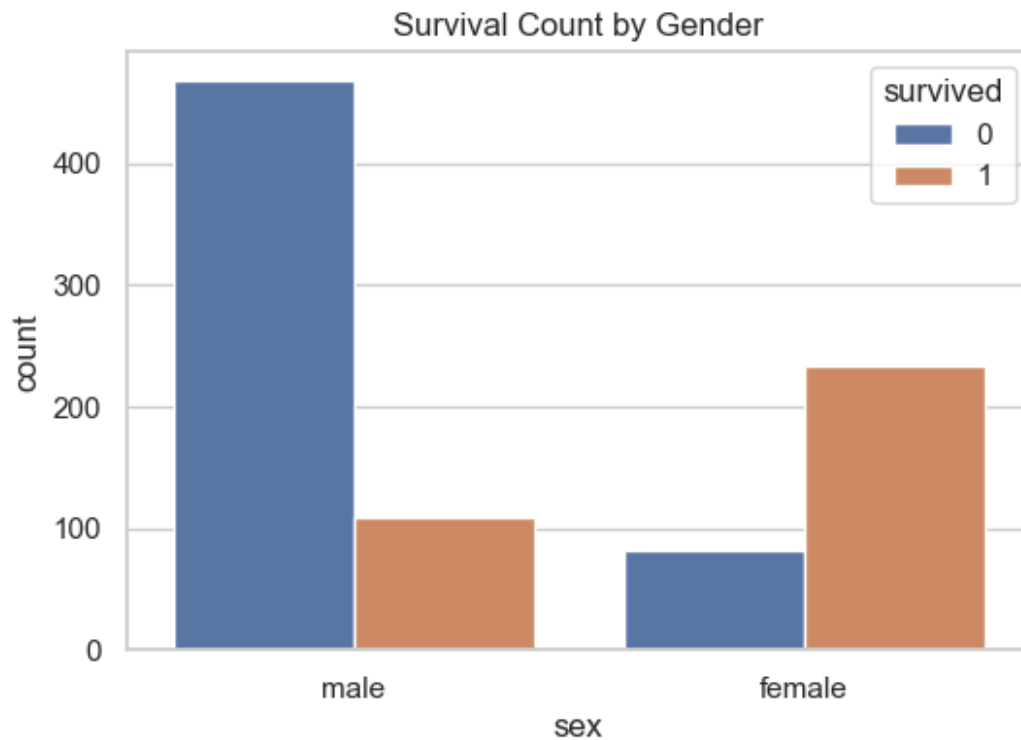
sns.countplot(x='pclass', data=titanic, ax=axes[1, 0])
axes[1, 0].set_title('Passenger Class Distribution')

sns.countplot(x='embarked', data=titanic, ax=axes[1, 1])
axes[1, 1].set_title('Port of Embarkation Distribution')

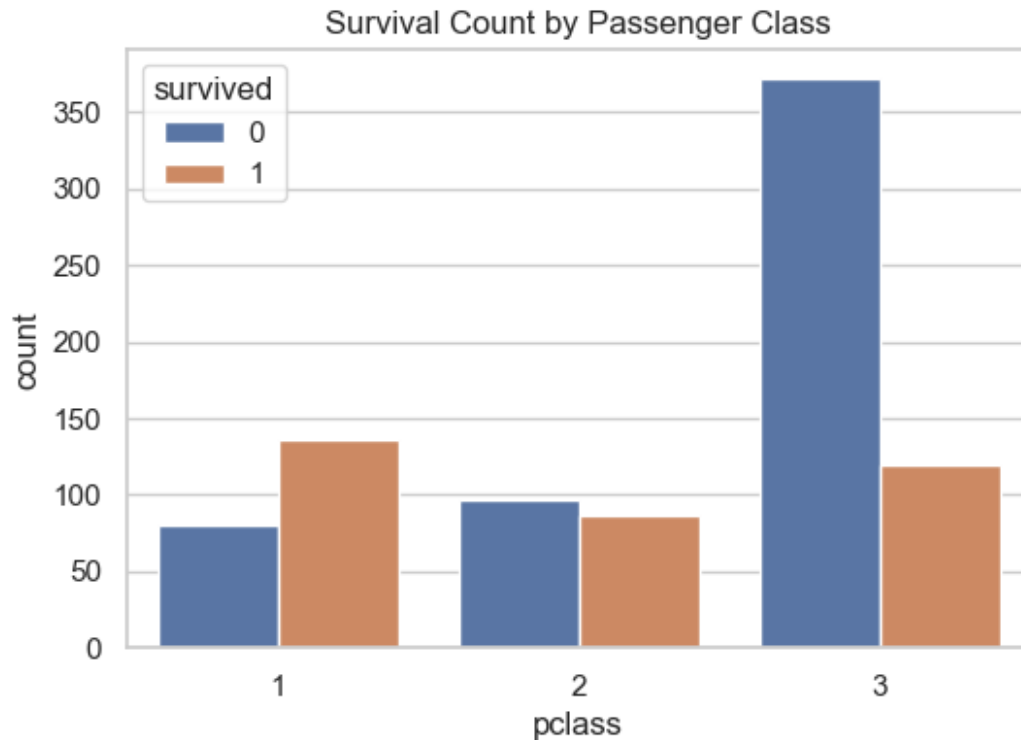
plt.tight_layout()
plt.show()
```



```
# Survival count grouped by gender
plt.figure(figsize=(6, 4))
sns.countplot(x='sex', hue='survived', data=titanic)
plt.title('Survival Count by Gender')
plt.show()
```



```
# Survival count grouped by passenger class  
plt.figure(figsize=(6, 4))  
sns.countplot(x='pclass', hue='survived', data=titanic)  
plt.title('Survival Count by Passenger Class')  
plt.show()
```



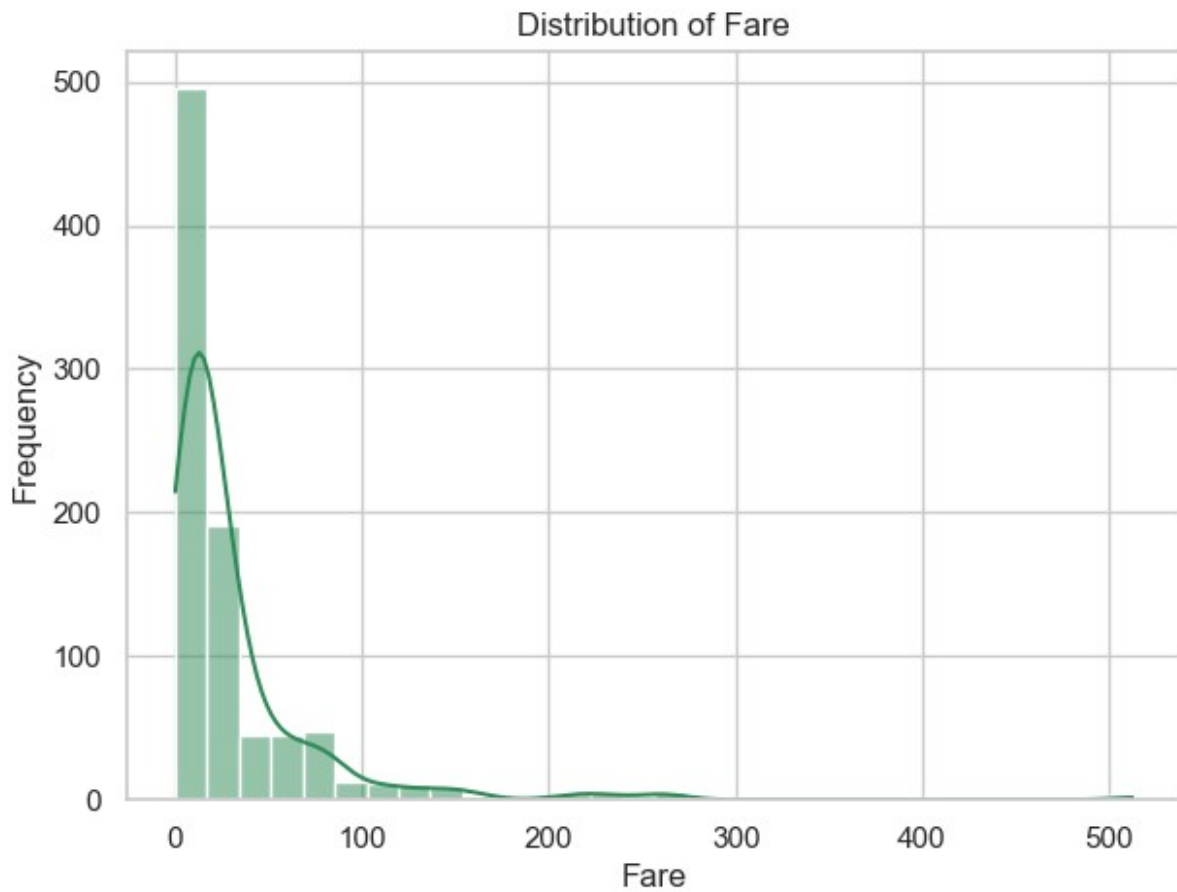
Step 7: Visualizing Numerical Variables

Histograms show the distribution/frequency of numerical variables, while box plots help identify outliers.

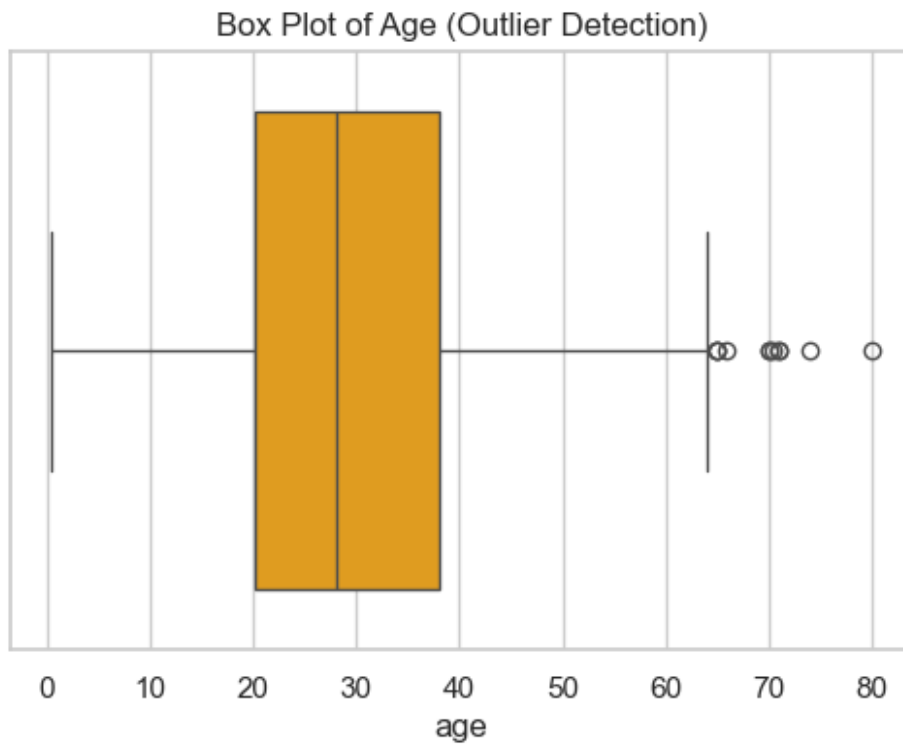
```
# Histogram of Age
plt.figure(figsize=(7, 5))
sns.histplot(titanic['age'].dropna(), bins=30, kde=True,
color='steelblue')
plt.title('Distribution of Age')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



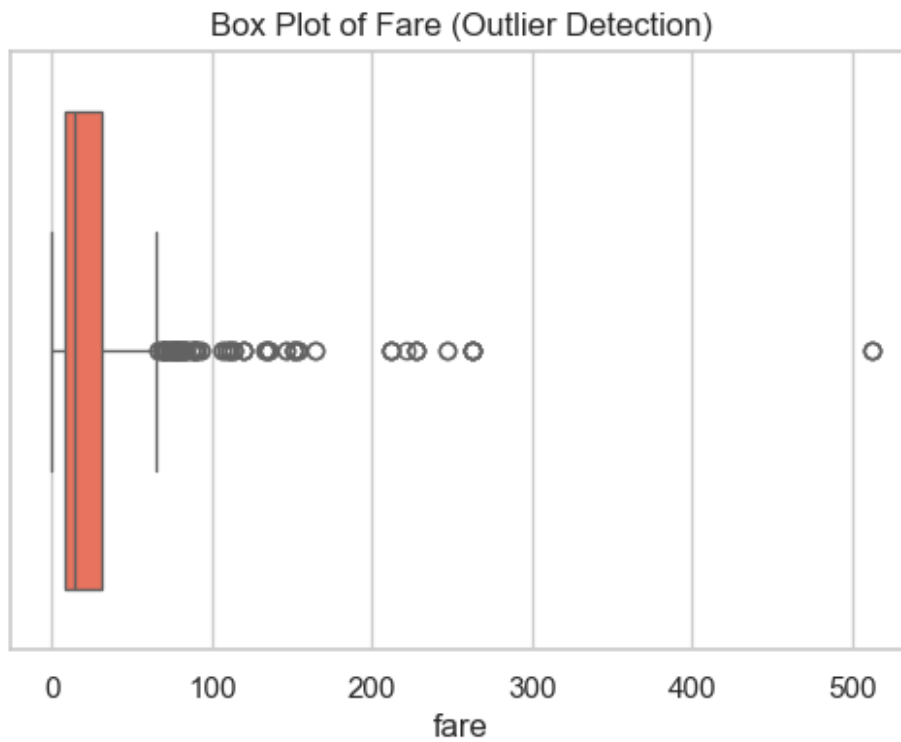
```
# Histogram of Fare
plt.figure(figsize=(7, 5))
sns.histplot(titanic['fare'].dropna(), bins=30, kde=True,
color='seagreen')
plt.title('Distribution of Fare')
plt.xlabel('Fare')
plt.ylabel('Frequency')
plt.show()
```



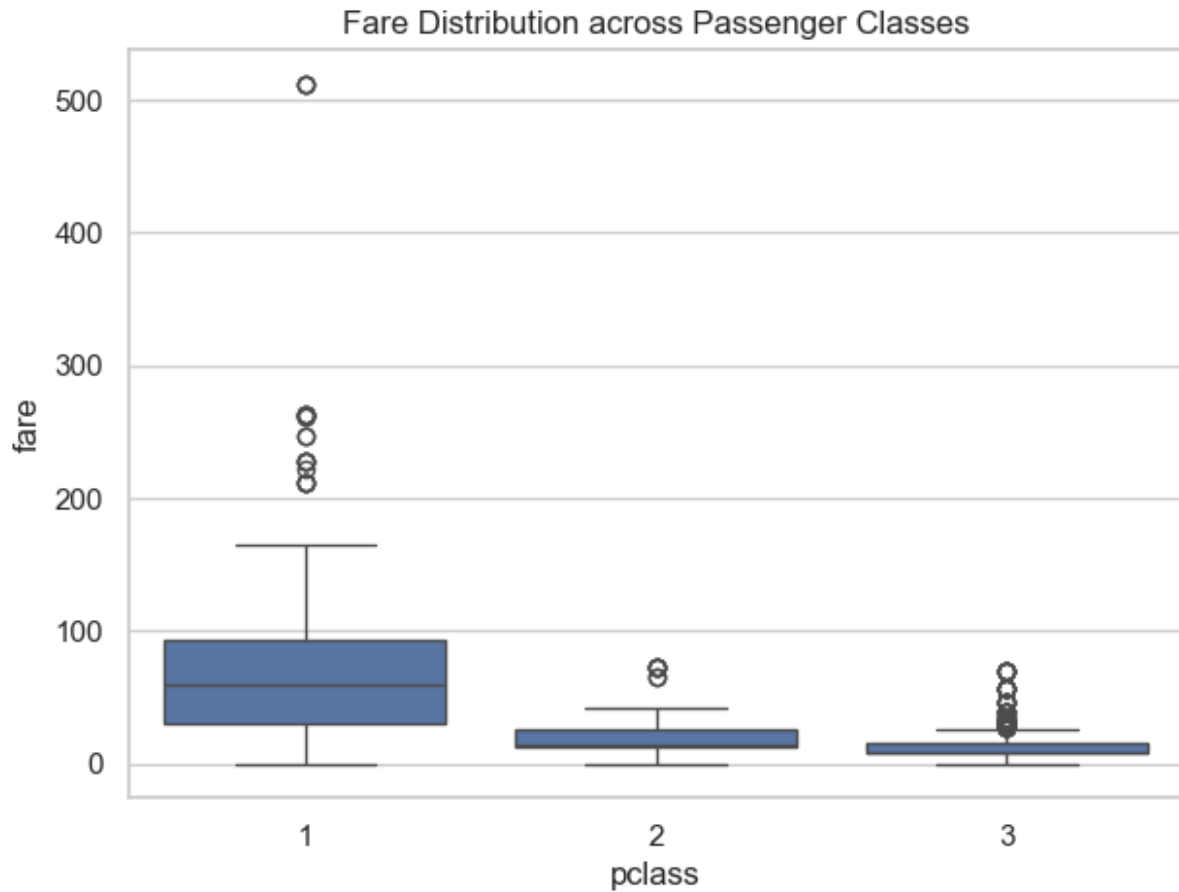
```
# Box plot for Age to detect outliers
plt.figure(figsize=(6, 4))
sns.boxplot(x=titanic['age'], color='orange')
plt.title('Box Plot of Age (Outlier Detection)')
plt.show()
```



```
# Box plot for Fare to detect outliers
plt.figure(figsize=(6, 4))
sns.boxplot(x=titanic['fare'], color='tomato')
plt.title('Box Plot of Fare (Outlier Detection)')
plt.show()
```



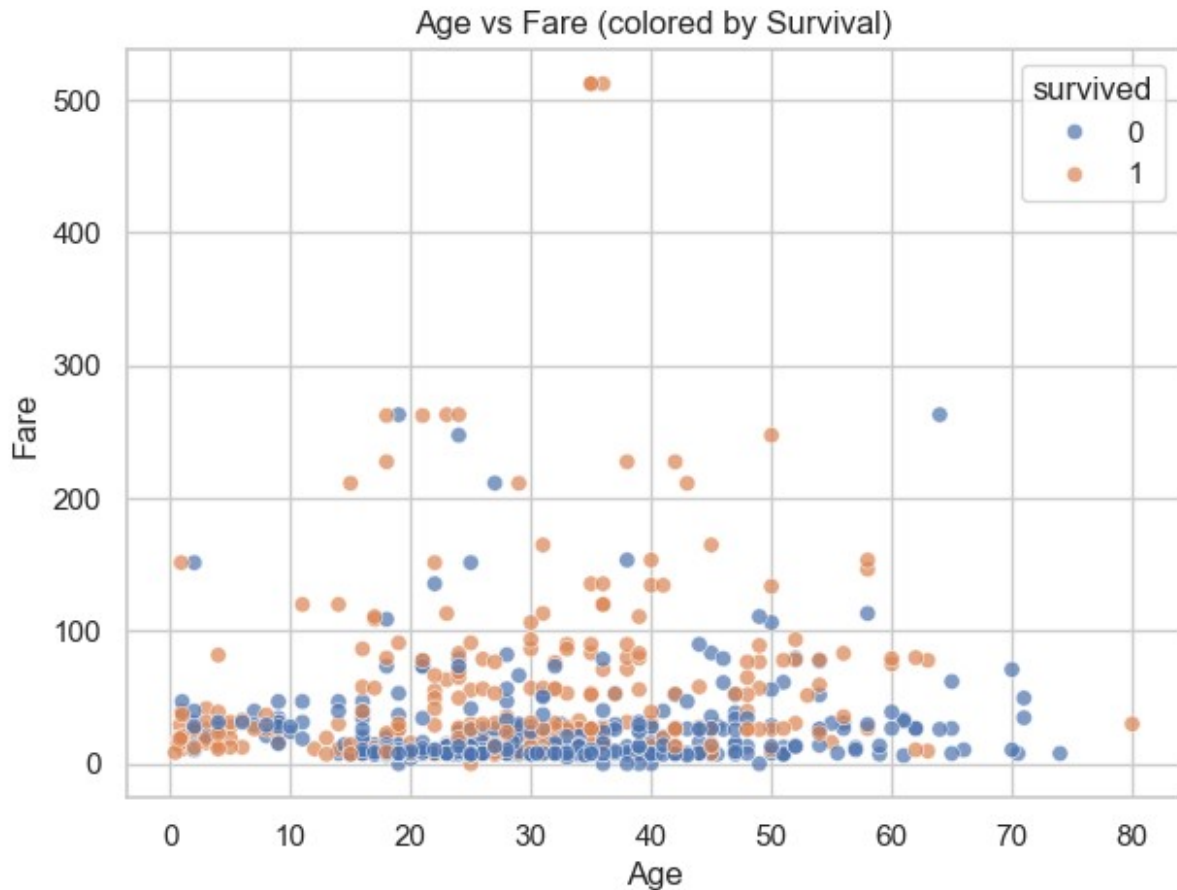
```
# Box plot of Fare grouped by Passenger Class
plt.figure(figsize=(7, 5))
sns.boxplot(x='pclass', y='fare', data=titanic)
plt.title('Fare Distribution across Passenger Classes')
plt.show()
```



Step 8: Scatter Plot

Scatter plots help visualize relationships between two numerical variables, such as Age and Fare.

```
plt.figure(figsize=(7, 5))
sns.scatterplot(x='age', y='fare', hue='survived', data=titanic,
alpha=0.7)
plt.title('Age vs Fare (colored by Survival)')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```

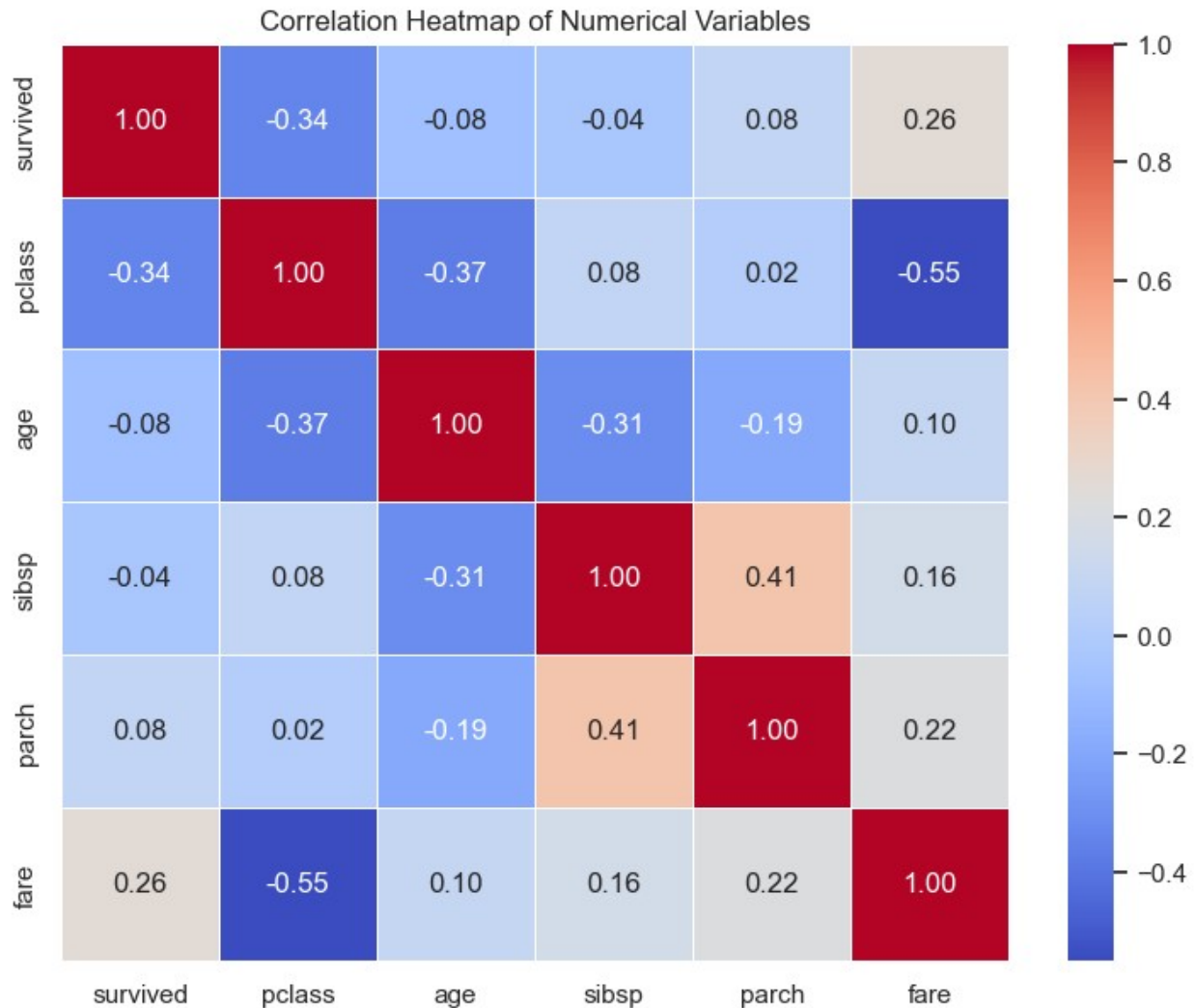


Step 9: Correlation Heatmap

A correlation heatmap displays the strength of the linear relationship between numerical variables. Values close to +1 indicate strong positive correlation, values close to -1 indicate strong negative correlation, and values near 0 indicate little to no correlation.

```
# Select only numerical columns for correlation
numeric_cols = titanic.select_dtypes(include=[np.number])

plt.figure(figsize=(9, 7))
corr_matrix = numeric_cols.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f',
            linewidths=0.5)
plt.title('Correlation Heatmap of Numerical Variables')
plt.show()
```



Step 10: Interpretation of Observations

- The **survival count** shows that fewer passengers survived than those who did not.
- **Gender** played a significant role in survival — a higher proportion of female passengers survived compared to male passengers.
- **Passenger class** also influenced survival — 1st class passengers had a noticeably higher survival rate than 3rd class passengers.
- The **Age** distribution is roughly right-skewed, with the majority of passengers between 20–40 years old.
- The **Fare** distribution is heavily right-skewed, with most fares concentrated at lower values and a few very high-value outliers.
- **Box plots** for Age and Fare reveal several outliers, especially in the Fare column, corresponding to passengers who paid unusually high ticket prices.
- The **correlation heatmap** shows a moderate positive correlation between **fare** and **survived**, and a negative correlation between **pclass** and **fare** (since class 1 = highest fare).

Experimental Results

Observation

- The Titanic dataset was successfully loaded and explored.
- Descriptive statistics summarized the numerical features of the dataset.
- Missing values were identified in some columns (mainly age, embarked, and deck).
- Histograms showed the distribution of Age and Fare.
- Box plots helped identify outliers.
- Heatmaps displayed the correlation among numerical variables.
- Data visualizations made it easier to understand passenger characteristics and survival patterns.

Result

Exploratory Data Analysis, descriptive statistical analysis, and data visualization were successfully performed using Python. The analysis provided useful insights into the Titanic dataset and demonstrated the importance of visualizing data before further analysis.

Conclusion

The experiment demonstrated the use of Python libraries for performing Exploratory Data Analysis (EDA), descriptive statistics, and data visualization. Various statistical measures and graphical techniques helped understand the structure, distribution, and relationships within the Titanic dataset. EDA serves as an essential step in the data science process by improving data understanding and supporting informed decision-making.

Exercises / Assignments

1. Calculate descriptive statistics for the Fare column.
2. Create a histogram of the Age variable.
3. Draw a box plot for Fare to identify outliers.
4. Generate a heatmap showing the correlation between numerical variables.
5. Interpret any one visualization created during the experiment.

The solutions to these exercises are demonstrated below.

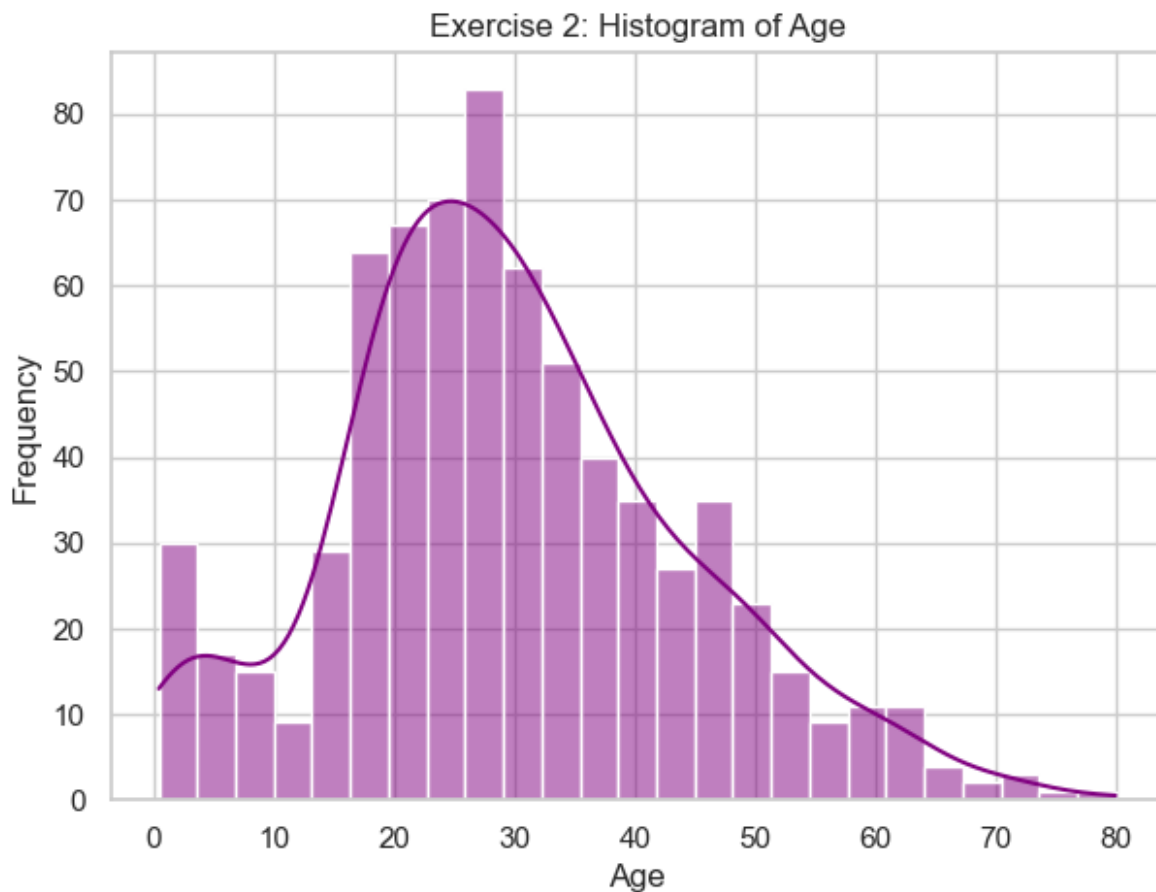
```
# Exercise 1: Descriptive statistics for the Fare column
print("Descriptive Statistics for Fare:\n")
print(titanic['fare'].describe())
```

Descriptive Statistics for Fare:

count	891.000000
mean	32.204208
std	49.693429
min	0.000000
25%	7.910400
50%	14.454200

```
75%      31.000000
max      512.329200
Name: fare, dtype: float64
```

```
# Exercise 2: Histogram of the Age variable
plt.figure(figsize=(7, 5))
sns.histplot(titanic['age'].dropna(), bins=25, kde=True,
color='purple')
plt.title('Exercise 2: Histogram of Age')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



```
# Exercise 3: Box plot for Fare to identify outliers
plt.figure(figsize=(6, 4))
sns.boxplot(x=titanic['fare'], color='gold')
plt.title('Exercise 3: Box Plot of Fare')
plt.show()
```

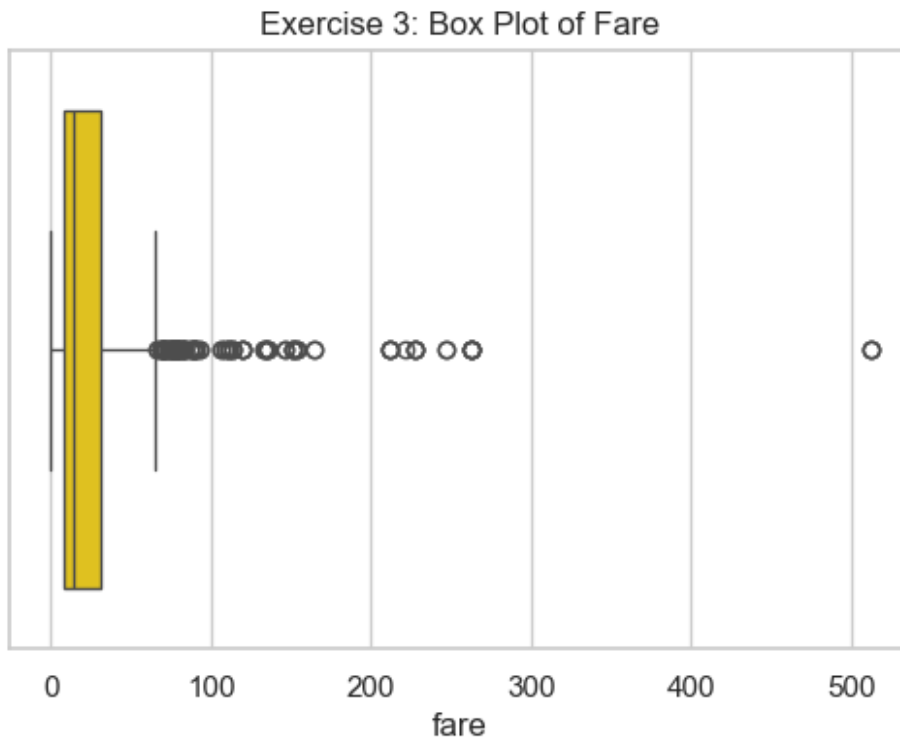
```
# Identify outliers using the IQR method
Q1 = titanic['fare'].quantile(0.25)
```

```

Q3 = titanic['fare'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = titanic[(titanic['fare'] < lower_bound) | (titanic['fare']
> upper_bound)]
print(f"Number of outliers in Fare: {len(outliers)}")

```

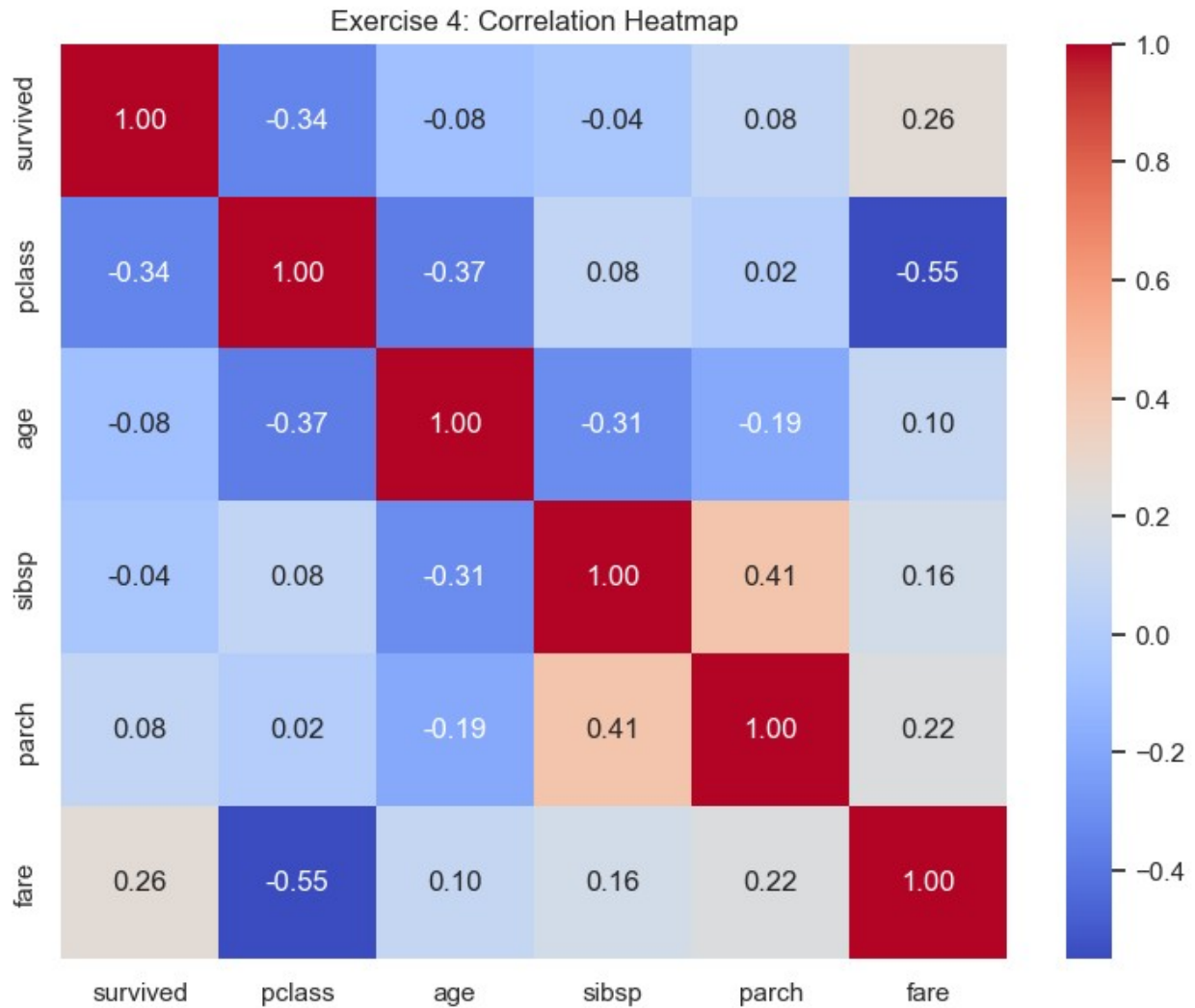


Number of outliers in Fare: 116

```

# Exercise 4: Heatmap showing correlation between numerical variables
plt.figure(figsize=(9, 7))
sns.heatmap(titanic.select_dtypes(include=[np.number]).corr(),
annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Exercise 4: Correlation Heatmap')
plt.show()

```



Exercise 5: Interpretation

Looking at the Fare box plot (Exercise 3), most passengers paid a relatively low fare (concentrated between roughly \$7 and \$31), but there are several outliers with very high fares (above \$100, with a few exceeding \$500). This right-skew indicates that a small number of wealthy first-class passengers paid disproportionately high fares compared to the rest of the passengers, which is consistent with the class-based pricing structure aboard the Titanic.